Doc. no.:	P2136R2
Date:	2020-12-5
Audience:	LWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

`invoke_r`

Changes Since R1

- update wording

Changes Since R0

- rename to `invoke_r`; the proposed facility is no longer overloaded with `std::invoke`

Introduction

This paper proposes `invoke_r`, a variant of `std::invoke` that allows specifying the return type, realizing the semantics of *`INVOKE`*`<R>` rather than *`INVOKE`*.

History

When `std::invoke` was introduced in N4169[1], `invoke<R>` was dropped from the proposal with the belief that such a form is unnecessary, stating:

> Mentioned version is leftover from TR1 implementation, were the result type was determined using `result_of` protocol or has to be specified at the call side and, after the introduction of type interference in C++11, it becomes obsolete.

But in 2015, LWG 2420[2] was applied to the working draft. *`INVOKE`*`(f, args…, void)` 's capability of casting away the return value is confirmed and specified.

In 2016, the author of this paper opened LWG 2690[3], proposing `std::invoke<R>`. The author of N4169 commented on that issue, confirmed that:

> The lack of `invoke<R>` was basically a result of the concurrent publication of the never revision of the paper and additional special semantics of *`INVOKE`*`(f, args…, void)`.

In 2017, *`INVOKE`*`(f, args…, void)` gained the current spelling *`INVOKE`*`<R>(f, args…)` in P0604R0[4]. In the same paper, all the new invocation traits get `_r` variants that allow specifying the return type.

In 2018, `std::visit<R>` [5] is added to the working draft. The usefulness of *`INVOKE`*`<R>` keeps getting attention.

Discussion

How useful `invoke_r` is?

`invoke_r<R>(...)` does three things that `std::invoke(...)` does not:

1. In a call forwarder that allows specifying the return type or the full signature, putting `void` as the return type naturally discards the return value, as implied by `std::is_invocable_r` and `std::is_nothrow_invocable_r`.
2. When `R` is not *cv* `void`, you can specify a compatible return type that is different from the callable entity. For example, you can request a function that returns `T&&` to return a prvalue of type `T` by calling `invoke_r<T>`.
3. If the callable entity has overloaded call operators that may return different types, they may agree on a return type that allows you to specify. For example, you can perform an upcast for covariant return types.

Why not to spell it `invoke<R>`?

LEWG reached consensus to name the facility differently to avoid ambiguity when `R` is the callable type, for the following reasons:

1. `std::invoke_r<R>` meant to be a low-level facility, therefore it supposes to be usable in any condition, unlike `std::bind<R>` and `std::visit<R>`.
2. Being consistent with an internal name for library specification, i.e., *`INVOKE`*`<R>`, isn't nessasry.
3. `invoke` with an explicit return type shouldn't belong to the `std::invoke` overload set, since an end-user never meant to get a different form when spelled a particular one.

To not to miss any bikeshedding opportunity, our naming candidates include:

1. `invoke_r`

2. `invoke_argh`

The author prefers (1) because users may correctly correlate `std::invoke_r`, `std::is_invocable_r`, and `std::is_nothrow_invocable_r` given the similarity.

Wording

The wording is relative to N4868.

Modify 20.14.2 [functional.syn], header `<functional>` synopsis, as indicated:

```
namespace std {
  // 20.14.5 [func.invoke], invoke
  template<class F, class... Args>
    constexpr invoke_result_t<F, Args...>   invoke(F&& f, Args&&... args)
      noexcept(is_nothrow_invocable_v<F, Args...>);
  template <class R, class F, class... Args>
    constexpr R invoke_r(F&& f, Args&&... args)
      noexcept(is_nothrow_invocable_r_v<R, F, Args...>);
```

Add the following sequence of paragraphs after 20.14.5 [func.invoke]/1 as indicated:

```
template <class R, class F, class... Args>
  constexpr R invoke_r(F&& f, Args&&... args)
    noexcept(is_nothrow_invocable_r_v<R, F, Args...>);
```

> *Constraints:* `is_invocable_r_v<R, F, Args...>` is `true`.
>
> *Returns:* *`INVOKE`*`<R>(std::forward<F>(f), std::forward<Args>(args)...)` (20.14.4 [func.require]).

References

1. Kamiński, Tomasz. *A proposal to add invoke function template (Revision 1)*. http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4169.html (http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4169.html) ↩

2. Bergé, Agustín. LWG 2420 `function<void(ArgTypes...)>` *does not discard the return value of the target object*. https://cplusplus.github.io/LWG/issue2420 (https://cplusplus.github.io/LWG/issue2420) ↩

3. Yuan, Zhihao. LWG 2690 `invoke<R>` . https://cplusplus.github.io/LWG/lwg-active.html#2690 (https://cplusplus.github.io/LWG/lwg-active.html#2690) ↩

4. Krügler, Daniel et al. *Resolving GB 55, US 84, US 85, US 86*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0604r0.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0604r0.html) ↩

5. Park, Michael and Agustín Bergé. `visit<R>` *: Explicit Return Type for* `visit` . http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0655r1.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0655r1.pdf) ↩